

Performance Analysis of Edge TPU Accelerators for Federated Learning on Constrained Devices

I. INTRODUCTION

The rapid advancement of Artificial Intelligence (AI) has led to an explosive demand for data to train machine learning (ML) models. A substantial portion of this data is collected/produced at the edge, made possible by the proliferation of billions of deployed IoT devices. Traditionally, raw data from these edge devices is transmitted to a centralized data center, where models are trained and later deployed for downstream inference tasks. However, this centralized paradigm faces two major concerns: (1) it places heavy strain on communication bandwidth to the data center due to the massive volume of transmitted data, and (2) it poses serious privacy risks, as sensitive user information (e.g., biometric data from smartwatches or network traffic from home routers) is exposed to the data collecting company.

To reconcile these issues, Federated Learning (FL) [17] has emerged as a promising approach that enables collaborative machine learning while preserving user privacy. At a high level as shown in Figure 1, FL operates in multiple rounds where each round performs the following: (1) given a global model, each client device updates it locally using its private data, (2) the locally updated model parameters (or gradients) are then sent to a central server, and (3) the server aggregates (e.g., by averaging) updates from all clients to produce a new global model, which is then (4) sent back to the devices for the next round.

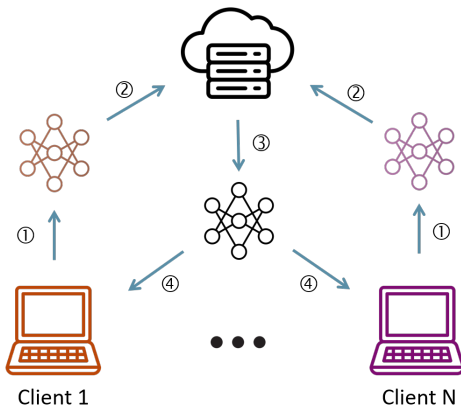


Fig. 1. Federated Learning

A large body of FL research focuses on high-end environments, where client devices are powerful servers or desktops equipped with medium/high-cost CPUs or GPUs. This makes techniques developed for such settings inapplicable to

resource-constrained edge devices that lack the computational power, memory, and bandwidth of higher-end machines. To enable FL on edge devices, prior work has explored several directions, such as compressing/simplifying model architectures to fit smaller memory footprints [28], [32], optimizing training code to reduce flash storage or RAM usage [13], [33], and minimizing communication overhead to accommodate limited bandwidth in the edge settings [3], [11].

Since hardware accelerators (predominantly GPUs) have played a critical role in enabling practical machine learning (including FL) on high-end systems, we believe that edge-based FL can equally benefit from hardware acceleration. While it is unrealistic to assume high-end GPUs on edge devices, recent advancements have introduced low-cost, production-ready ML accelerators, such as micro Neural Processing Units (microNPUs), e.g., ARM's Ethos-U65 [1], as well as Edge TPUs [26], specifically designed for resource-constrained IoT environments. In existing literature [21], [25], [23], [26], these accelerators have primarily been used to speed up inference tasks, typically under the assumption that model training happened earlier in the centralized fashion [16]. To the best of our knowledge, no prior work has examined the potential of these low-cost accelerators to support FL operations on edge devices or quantified the resulting performance gains.

In this work, we present the first study that explores the integration of low-cost hardware accelerators into edge-based FL. Specifically, we focus on *Edge TPUs* [9] chosen due to their commercial availability and mature software ecosystem. Our integration, called FL-ETPU, builds upon the prior technique [16] that introduces a lightweight training pipeline designed to support on-device FL operations using CPU. To understand the impact of Edge TPUs in FL, we conduct extensive experimental evaluation, comparing the performance of FL-ETPU with a CPU-only baseline (FL-CPU).

We anticipate the following contributions in this work:

- 1) We propose FL-ETPU, an approach that integrates edge-based FL with Edge TPUs. FL-ETPU builds on a prior technique that enables efficient on-device FL by (i) leveraging a pre-trained base model (e.g., convolution layers) and (ii) training/sharing only the relatively small classification head (e.g., softmax layers). We identify that the first component can be accelerated using Edge TPU and propose to modify the training pipeline to offload this operation to Edge TPU.
- 2) We conduct a detailed benchmarking analysis of FL-ETPU and compare it with a CPU-only baseline (FL-

CPU) across three key performance metrics: runtime, peak memory usage, and energy consumption. To measure its performance on devices with varying resources, FL-ETPU was evaluated on two edge development boards: Raspberry Pi 3A+ and Raspberry Pi 5.

3) Our experimental results indicate that: TODO

II. RELATED WORK

A. Inference on Constrained Edge Devices

Prior studies have developed various optimization methods that enable efficient inference on resource-constrained devices. A recent survey [30] categorizes these methods into four groups: (1) designing lightweight models that use custom modules with fewer parameters and lower computational requirements, such as depthwise separable convolution (as proposed in MobileNetV2 [24]) and group convolution (used in AlexNet [12] and CondenseNet-86 [10]); (2) compressing existing models using techniques such as parameter pruning [28] and low-bit quantization [32] to reduce model size and computational complexity while maintaining reasonable accuracy; (3) applying compilation-based techniques [18], [31] to optimize the model's computation graph and efficiently transform the optimized version into executable binaries tailored for specific hardware platforms, which improves execution efficiency and reduces resource consumption of the model on edge devices and, (4) leveraging multiple devices to perform collaborative inference [5], [15], which distributes computation across several edge devices to enhance both inference accuracy and efficiency.

B. On-device Training and FL at the Edge

Compared to edge inference, training locally on constrained edge devices has received relatively little attention. One research direction addresses this challenge by formulating resource limitations as part of an optimization problem during training [34]. To minimize memory usage, Tiny Training Engine (TTE) [13] proposes to skip gradient computations for less important model parameters. This is achieved by considering an optimization problem based on the contribution of each parameter, which is then solved to identify and perform a training on a subset of parameters with maximized overall contribution within a given memory budget. POET [19] views the backpropagation process as an optimization problem that accounts for both computation cost and training memory usage [4], [20]. It then solves this optimization problem to find optimal training schedule with minimal energy consumption for a given memory budget.

A line of work introduce several techniques to enable FL on constrained devices, commonly by training only a part of the model [14], [16], [29], [22]. Among these methods, the approach in [16] is most closely related to our work. It proposes an on-device FL framework that splits the model into two components: (1) a pre-trained base model, which is centrally trained and distributed to edge devices, and (2) a classification head, which is locally trained and then shared to/aggregated by the server during the FL process. Since

the classification head is substantially smaller than the base model, this design significantly reduces memory and energy consumption throughout the entire FL pipeline. However, the evaluation in [16] is conducted by running all operations using only CPU; in this work, we show that integrating this approach with Edge TPU can further reduce computation costs and energy consumption, making it even more suitable for energy-constrained edge devices.

C. Hardware Accelerators on Edge Devices

Hardware accelerators have become essential for improving the performance of machine learning workloads on edge devices. A widely adopted example is the Coral Edge TPU [26], which has been extensively used in both vision-related [21], [25] and text-related [23] applications. The most relevant work that integrates the Edge TPU into federated learning (FL) is presented in [8], where only the upper layers of a model are fine-tuned on the Edge TPU, resulting in improved local accuracy compared to using the pre-trained baseline model. In contrast, our work also uses the Edge TPU to train the classification head (upper layers); however, rather than focusing on accuracy improvements, we aim to evaluate the resource efficiency of the Edge TPU, specifically in terms of computation time and energy consumption, under edge-device constraints.

Beyond the Edge TPU, other low-cost accelerators such as micro NPUs (e.g., Ethos-U65 [1]), eIQ [2], and various custom hardware designs [27], [7], [6] have been explored in the literature to accelerate ML workloads on edge devices. However, to the best of our knowledge, these accelerators have not yet been adopted in FL settings.

III. FL-ETPU: FEDERATED LEARNING WITH EDGE TPU

The main challenge of applying Edge TPU to FL is that it can only accelerate the inference (forward-pass) process and does not natively support the actual training operations, i.e., backpropagation [26]. Consequently, FL clients cannot use the Edge TPU for computing local model updates and must rely on the CPU instead.

To still benefit from Edge TPU in FL, we adopt the method from prior work [16] that minimizes client-side training in FL by splitting a model into two parts: a base model and a classification head. The base model, which constitutes most of the network, is pre-trained on a powerful server and then deployed to client devices, where it is frozen (i.e., kept untrainable) and used only as a feature extractor. This leaves each FL client to train only the relatively lightweight classification head, typically consisting of one or two layers, which can be efficiently handled on the CPU.

Following this idea, FL-ETPU operates in two phases as shown in Figure 2: an offline phase and a runtime phase. The offline phase runs before FL starts, where the server first obtains a pre-trained base model, freezes it, and distributes it to all clients (step 1). Each client then loads the received base model into its Edge TPU (step 2) and applies it to its local dataset to extract embeddings (step 3).

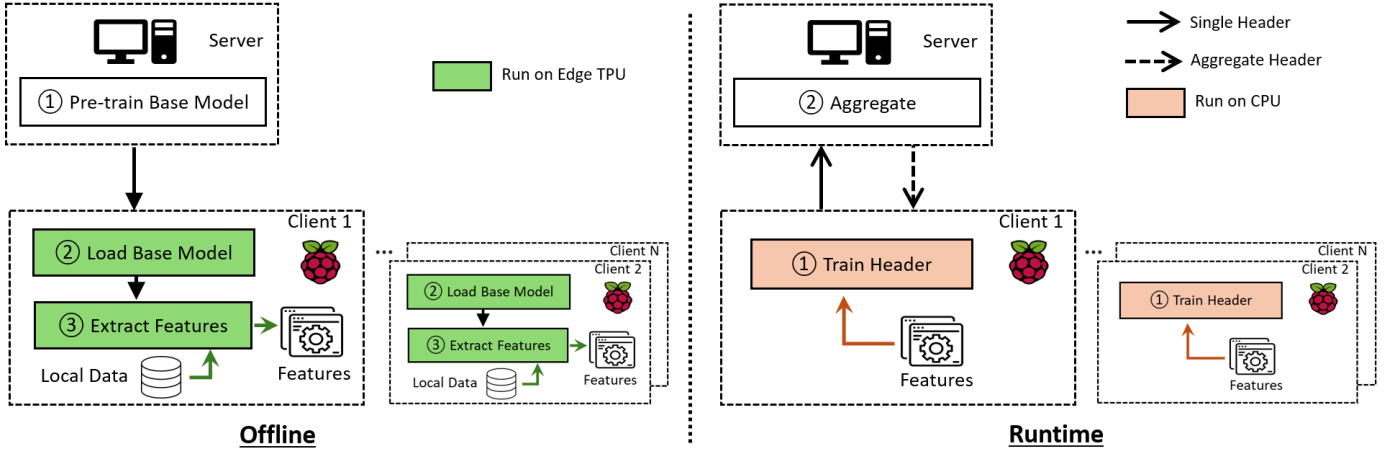


Fig. 2. FL-ETPU workflow

Once FL starts, FL-ETPU triggers the runtime phase: the server initializes a classification head with random weights and distributes it to clients at the start of training (step 4). Upon receiving the head (step 5), each client trains it using the pre-extracted features and returns the updated parameters for the classification head to the server. The server then aggregates all client updates, e.g., via weighted averaging in FedAVG [17], and redistributes the aggregate classification head for the next training round (step 6).

Notably, the key distinction between FL-ETPU and the method proposed in [16] is that we recognize that while the local updates cannot be accelerated by the Edge TPU, the feature extraction operation can. Accordingly, FL-ETPU leverages the Edge TPU for this operation, whereas [16] performs all FL computations solely on the CPU. All client-side operations in FL-ETPU, including whether they are executed on the CPU or Edge TPU, are detailed in Algorithm 1. For comparison, we denote the CPU-only method in [16] as the baseline approach, referred to as FL-CPU. In the next sections, we evaluate the performance of FL-ETPU and FL-CPU to quantify the benefits introduced by using Edge TPU in FL.

IV. EXPERIMENTAL SETUP

A. Implementation

We implement both FL-ETPU and FL-CPU in Python on top of the Flower federated learning framework [?]. Flower offers a standardized FL workflow built around two core functions: `fit` and `aggregate`. The `fit` function runs on a Flower client and defines the procedure for local training updated on the client's local dataset. It outputs the updated model and the number of samples trained, which Flower automatically forwards to the server. Once the server receives results from all clients, Flower calls `aggregate` on the Flower server, which applies the aggregation logic to produce a combined model. The combined model is then sent back to the clients to begin the next training round.

To implement FL-ETPU, we leave the default `aggregate` function unmodified, where it performs aggregation of local

Algorithm 1: Client-side operations in FL-ETPU

Offline Phase (run before FL starts)
Input: Pre-trained base model B (from the server) and a local dataset D
Output: Extracted embeddings E (kept locally)
// Run on Edge TPU
1 $\text{LOAD_MODEL}(B)$
2 $E \leftarrow \text{EXTRACT_FEATURES}(B; D)$

Runtime Phase (run for each FL round)
Input: Classification head H (from the server), embeddings E (from Phase 1)
Output: Updated classification head H' (sent to the server)
// Run on CPU
3 $H' \leftarrow \text{LOCAL_UPDATE}(H; E)$

models using the standard FedAVG's weighted averaging strategy. We implement the FL-ETPU offline phase during the FL client's initialization. It takes an Edge TPU-compatible base model as input and uses the PyCoral APIs to execute the first two lines from Algorithm 1 on the Edge TPU. In contrast, for FL-CPU, the offline phase uses a TensorFlow Lite¹ model as input and performs the client's operations on CPU using the TensorFlow Lite APIs. Both FL-ETPU and FL-CPU implements the runtime phase the same way, inside the `fit` function, which performs a local update on the classification head using standard TensorFlow APIs and return the updated classification head to the server.

B. Metrics

To evaluate the performance benefits of the Edge TPU, we consider three metrics relevant to the critical resource usage in edge devices: *execution time*, *peak RAM usage*, and

¹TensorFlow Lite is a lightweight version of TensorFlow, commonly deployed on edge devices.

power consumption. We measure execution time using the standard `time` Python API, while peak memory usage per client-side operation is recorded with Memray². Finally, we monitor power consumption by connecting an FNB58 tester³ between the edge development board and its power source; the experimental setup is shown in Figure 3.

C. Benchmarking Setup

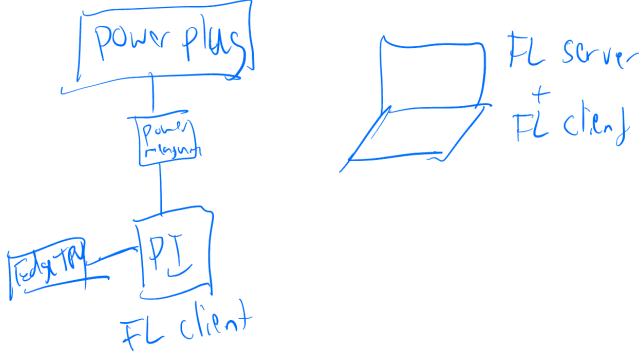


Fig. 3. Benchmarking Setup. See Fig. 6 in <https://openreview.net/pdf/648914b56e5c4dc27b0a2905008b88750a48bfe1.pdf>

For benchmarking performance of FL-ETPU and FL-CPU, we create an experimental setup with one server and two FL clients (one of which is an edge embedded board). The number of clients is selected to a minimum (which is 2 in FL) because our goal is to measure on-device performance on an FL edge client, which is independent of other FL clients.

Figure 3 illustrates our actual benchmarking setup: one client runs on an edge embedded board connected to an Edge TPU USB accelerator, while the server is hosted on a MacBook and the second client is simulated on the same MacBook. To explore different levels of resource constraints, we conduct benchmarking on two types of embedded boards: Raspberry Pi 3B+ and Raspberry Pi 5, where the latter represents a less constrained device with a faster CPU (2.4 GHz vs. 1.4 GHz) and larger RAM (4 GB vs. 1 GB). We use WLAN for communication between the server and the embedded board. For FL-CPU, we also evaluate the performance impact under multi-core settings where we experiment with 1, 2, and 4 CPU cores.

D. Datasets and Models

Unless specified otherwise, our benchmarking uses 1,xxx RGB images for local training on the embedded board client and considers two CNN models: MobileNetV1 [?] (with 4M parameters) and InceptionV3 [?] (with 52M parameters). For each model, we treat the last layer (i.e., the fully-connected softmax layer) as the classification head, which is trained in each FL round (line 3 of Algorithm 1), and the remaining

layers as the base model, which is used in the offline phase as a feature extractor (lines 1 and 2).

V. RESULTS

Texts

TABLE I
EXPERIMENTS ON MOBILENET MODELS

No.	Model Description	Model File Name	Runtime			Peak Memory (MB)	CPU Utilization (%)	Energy Consumption
			Loading Feature Extractor	Performing Feature Extractor	Training Classification Head			
1.	Model on Edge TPU	mobilenet_v1_1.0_224_quant_embedding_extractor_edgetpu.offline						
2.	Model on CPU (1 core)	mobilenet_v1_1.0_224_quant_embedding_extractor.offline						

VI. CONCLUSION & FUTURE WORK

Federated Learning presents a promising paradigm for privacy-preserving collaborative machine learning, but its deployment on resource-constrained edge devices faces significant computational and energy challenges. While low-cost hardware accelerators like the Edge TPU have emerged to speed up inference tasks on such devices, their potential to assist in FL operations has remained largely unexplored, primarily because they do not natively support training procedures like backpropagation.

In this work, we introduced FL-ETPU, a practical approach to integrate commercially available Edge TPUs into the edge FL pipeline. By adopting a model-splitting technique where a pre-trained base model is used for feature extraction and only a lightweight classification head is trained locally, FL-ETPU effectively leverages the Edge TPU to accelerate the computationally intensive feature extraction phase, while the training of the head remains on the CPU.

Our main contributions are threefold: the proposal and implementation of the FL-ETPU integration using the Flower framework, a comprehensive performance evaluation quantifying the benefits of using the Edge TPU in this specific FL context, and the commitment to releasing our implementation to facilitate further research. This study confirms the feasibility and highlights the potential resource savings of strategically incorporating existing inference accelerators into edge-based Federated Learning workflows.

REFERENCES

- [1] Arm Ltd. Arm Ethos-U65 MicroNPU. <https://www.arm.com/products/silicon-ip-cpu/ethos/ethos-u65>, 2025. Accessed: 2025-08-04.
- [2] Lennart Bamberg, Filippo Minnella, Roberto Bosio, Fabrizio Ottati, Yuebin Wang, Jongmin Lee, Luciano Lavagno, and Adam Fuks. *eiq neutron: Redefining edge-ai inference with integrated npu and compiler innovations*. *arXiv preprint arXiv:2509.14388*, 2025.
- [3] Hao Chen, Shaocheng Huang, Deyou Zhang, Ming Xiao, Mikael Skoglund, and H Vincent Poor. Federated learning over wireless iot networks with optimized communication and resources. *IEEE Internet of Things Journal*, 9(17):16592–16605, 2022.
- [4] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [5] Yanming Chen, Tong Luo, Weiwei Fang, and Neal N Xiong. Edgeci: Distributed workload assignment and model partitioning for cnn inference on edge clusters. *ACM Transactions on Internet Technology*, 24(2):1–24, 2024.

²<https://github.com/bloomberg/memray>

³<https://www.fnirsi.com/products/fnb58>

- [6] Yunji Chen, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. Diannao family: energy-efficient hardware accelerators for machine learning. *Communications of the ACM*, 59(11):105–112, 2016.
- [7] Francesco Conti, Pasquale Davide Schiavone, and Luca Benini. Xnor neural engine: A hardware accelerator ip for 21.6-fj/op binary neural network inference. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(11):2940–2951, 2018.
- [8] Erik Dubois. Shared learning among distributed edge devices using coral edge tpu machine learning engines. 2021.
- [9] Google LLC. Products — Coral. <https://coral.ai/products/>, 2025. Accessed: 2025-08-04.
- [10] Gao Huang, Shichen Liu, Laurens Van der Maaten, and Kilian Q Weinberger. Condensenet: An efficient densenet using learned group convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2752–2761, 2018.
- [11] Yu Ji and Lan Chen. Fedqnn: A computation–communication-efficient federated learning framework for iot with low-bitwidth neural network quantization. *IEEE Internet of Things Journal*, 10(3):2494–2507, 2022.
- [12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [13] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. On-device training under 256kb memory. *Advances in Neural Information Processing Systems*, 35:22941–22954, 2022.
- [14] Nil Llisterra Gimenez, Marc Monfort Grau, Roger Pueyo Centelles, and Felix Freitag. On-device training of machine learning models on microcontrollers with federated learning. *Electronics*, 11(4):573, 2022.
- [15] May Malka, Erez Farhan, Hai Morgenstern, and Nir Shlezinger. Decentralized low-latency collaborative inference via ensembles on the edge. *IEEE Transactions on Wireless Communications*, 2024.
- [16] Akhil Mathur, Daniel J Beutel, Pedro Porto Buarque de Gusmao, Javier Fernandez-Marques, Taner Topal, Xinchu Qiu, Titouan Parcollet, Yan Gao, and Nicholas D Lane. On-device federated learning with flower. *arXiv preprint arXiv:2104.03042*, 2021.
- [17] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [18] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. Dnnfusion: accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 883–898, 2021.
- [19] Shishir G Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph Gonzalez. Poet: Training neural networks on tiny devices with integrated rematerialization and paging. In *International Conference on Machine Learning*, pages 17573–17583. PMLR, 2022.
- [20] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 891–905, 2020.
- [21] Ruben Prokscha, Mathias Schneider, and Alfred Höß. Efficient edge deployment demonstrated on yolov5 and coral edge tpu. In *Industrial Artificial Intelligence Technologies and Applications*, pages 141–155. River Publishers, 2023.
- [22] Xinchu Qiu, Javier Fernandez-Marques, Pedro PB Gusmao, Yan Gao, Titouan Parcollet, and Nicholas Donald Lane. Zerooff: Efficient on-device training for federated learning with local sparsity. *arXiv preprint arXiv:2208.02507*, 2022.
- [23] Brendan C Reidy, Mohammadreza Mohammadi, Mohammed E Elbity, and Ramtin Zand. Efficient deployment of transformer models on edge tpu accelerators: A real system evaluation. In *Architecture and System Support for Transformer Models (ASSYST@ ISCA 2023)*, 2023.
- [24] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.
- [25] Jonah Sengupta, Rajkumar Kubendran, Emre Neftci, and Andreas Andreou. High-speed, real-time, spike-based object tracking and path prediction on google edge tpu. In *2020 2nd IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, pages 134–135. IEEE, 2020.
- [26] Kiran Seshadri, Berkin Akin, James Laudon, Ravi Narayanaswami, and Amir Yazdanbakhsh. An evaluation of edge tpu accelerators for convolutional neural networks. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*, pages 79–91. IEEE, 2022.
- [27] Thierry Tambe, Coleman Hooper, Lillian Pentecost, Tianyu Jia, En-Yu Yang, Marco Donato, Victor Sanh, Paul Whatmough, Alexander M Rush, David Brooks, et al. Edgebert: Sentence-level energy optimizations for latency-aware multi-task nlp inference. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 830–844, 2021.
- [28] Changdi Yang, Pu Zhao, Yanyu Li, Wei Niu, Jiexiong Guan, Hao Tang, Minghai Qin, Bin Ren, Xue Lin, and Yanzhi Wang. Pruning parameterization with bi-level optimization for efficient semantic segmentation on the edge. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15402–15412, 2023.
- [29] Tien-Ju Yang, Dhruv Guliani, Françoise Beaufays, and Giovanni Motta. Partial variable training for efficient on-device federated learning. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4348–4352. IEEE, 2022.
- [30] Runhua Zhang, Hongxu Jiang, Wei Wang, and Jinhao Liu. Optimization methods, challenges, and opportunities for edge inference: A comprehensive survey. *Electronics*, 14(7):1345, 2025.
- [31] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. Ansor: Generating {High-Performance} tensor programs for deep learning. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, pages 863–879, 2020.
- [32] Qihua Zhou, Song Guo, Zhihao Qu, Jingcai Guo, Zhenda Xu, Jiawei Zhang, Tao Guo, Boyuan Luo, and Jingren Zhou. Octo-{INT8} training with loss-aware compensation and backward quantization for tiny on-device learning. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 177–191, 2021.
- [33] Ligeng Zhu, Lanxiang Hu, Ji Lin, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. Pockengine: Sparse and efficient fine-tuning in a pocket. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 1381–1394, 2023.
- [34] Shuai Zhu, Thiemo Voigt, Fatemeh Rahimian, and Jeonggil Ko. On-device training: A first overview on existing systems. *ACM transactions on sensor networks*, 20(6):1–39, 2024.